ORIGINAL ARTICLE

A genetic engineering algorithm for the generalized quadratic assignment problem

Majid Sohrabi[1,2] · Amir M. Fathollahi-Fard[3] · Vasilii A. Gromov[1] · Maxim A. Dulebenets[4]

Received: 3 December 2024 / Accepted: 26 February 2025 / Published online: 28 March 2025
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2025

Abstract
The generalized quadratic assignment problem (GQAP) poses a significant challenge in optimization, known for its NP-hard complexity and wide-ranging applications in supply chain and manufacturing contexts. While various metaheuristic algorithms have addressed the GQAP, this paper introduces a novel modification of the genetic algorithm (GA), namely the genetic engineering algorithm (GEA). A traditional GA initializes the population of solutions randomly and refines these solutions iteratively through crossover and mutation operators. However, its reliance on randomness often leads to premature convergence, susceptibility to local optima, and suboptimal solutions at termination of the algorithmic run. To address these drawbacks, our research presents a modified GA framework inspired by genetic engineering principles. This framework introduces novel search mechanisms mimicking gene mining processes, targeting population enhancement through insertion, purification, and genetic material exchange from elite solutions. The GEA adds three search scenarios to the traditional GA framework to exhibit desired characteristics from chromosomes, enhancing solution quality and convergence speed. This GEA demonstrates a superior performance across various search scenarios against state-of-the-art algorithms and exact solutions for the benchmark instances of the GQAP. Additionally, this study conducts comprehensive sensitivity analyses on the search scenarios of the GEA. By employing gene mining strategies, the proposed GEA offers promising solutions for effectively addressing the NP-hard optimization challenges of the GQAP on large-scale datasets.

Keywords Genetic algorithm · Metaheuristic algorithms · Optimization · Genetic engineering algorithm · Generalized quadratic assignment problem

1 Introduction and literature review

The generalized quadratic assignment problem (GQAP) presents a complex optimization problem [1]. Its complexity arises from the integration of two fundamental cost components, each crucial in elucidating the intricacies of the problem. Initially, there exist expenses linked to the assignment of particular machinery or equipment units to individual factories, which encapsulates factors of practicality and effectiveness during the allocation procedure. In addition, quadratic expenses represent the interplay among equipment units, encompassing the complex connections and interdependencies among various equipment units and the distance between the factories. Moreover, the GQAP imposes two primary real-life constraints: assignment constraints ensuring each piece of equipment is assigned to only one factory, and capacity limitations of these factories. The presence of both linear and quadratic terms in the objective function which is nonlinear, along with these complex constraints, renders the GQAP one of the significant challenges in combinatorial optimization [2].

By eliminating the quadratic term from the GQAP objective function, the resulting model becomes akin to the generalized assignment problem (GAP). Given that the GAP is recognized as an NP-hard optimization problem [3], it follows that the GQAP inherits a higher NP-hard complexity due to an additional nonlinear quadratic term. Lee and Ma [1] studied the GQAP complexity and conducted an analysis for approximate solutions in large-scale optimization problems. Because of its high complexity, a plethora of exact, heuristic, and metaheuristic algorithms have emerged in the GQAP literature [4]. As per the no-free-lunch theory [5], no metaheuristic algorithm demonstrates effectiveness for all combinatorial optimization problems. Hence, the GQAP literature abounds with the development of new modified and hybrid metaheuristic algorithms tailored for efficient problem-solving.

Metaheuristic algorithms, such as the genetic algorithm (GA), tabu search (TS) algorithm, and the greedy randomized adaptive search procedure (GRASP), have been commonly employed in the literature [6–9]. For instance, Mateus et al. [10] addressed the GQAP utilizing a hybrid approach comprising of GRASP with path-relinking local search, along with constructive heuristics. Liu et al. [11] extended the problem by introducing equilibrium constraints and proposed a GA based on an equilibrium heuristic to address it.

Billionnet et al. [12] solved various quadratic assignment problems, including the GQAP, by the quadratic convex reformulation (QCR) method. In a subsequent study, they [13] further refined this approach and developed a compact QCR (CQCR), which is an efficient equivalent quadratic approach using a convex reformulation. Robenek et al. [14] addressed an extended version of the GQAP pertaining to berth allocation and yard assignment using a branch-and-price algorithm. McKendall and Li [15] tackled the GQAP employing a TS metaheuristic algorithm augmented by constructive heuristics and local search techniques. They conducted a comparative analysis of its performance against that of the GRASP and the memetic algorithm (MA).

Chmiel [16] investigated the conditional expected value approach for the quadratic GQAP, utilizing various evolutionary algorithms with a focus on the GA with innovative pseudo-genetic operators. Bergman [17] developed a decomposition-based branch-and-price algorithm tailored for the GQAP, showcasing its effectiveness in solving complex seating arrangement applications, such as table event seating. Guignard [18] introduced two models of the reformulation linearization technique (RLT), with one model exhibiting superior performance in solving quadratic assignment problems. This technique demonstrated a notable effectiveness in addressing challenges like the cross-dock door assignment problem within supply chain networks. D'Ambrosio et al. [19] presented a novel variant of the GAP incorporating nonlinear constraints for capacity and assignment weights, offering constructive heuristics and a decomposition formulation to establish model boundaries. Silva et al. [20] conducted a comprehensive review of all the quadratic assignment problems. They [20] introduced a highly efficient parallel MA leveraging a TS algorithm for solving large-scale quadratic assignment problems. Furthermore, Zhou et al. [21] developed a hybrid evolutionary search algorithm that combines GA and TS, enhancing a lower bound for quadratic multiple knapsack problems.

More recently, Song and Cheng [22] presented a novel adaptation of the GAP by incorporating reliability constraints into the model. They proposed a Lagrangian relaxation approach employing a decomposition method to facilitate efficient solution derivation. Fathollahi-Fard et al. [2] proposed a set of heuristics and reformulations based on an adaptive large neighborhood search (ALNS) algorithm for solving the GQAP in both small and large data sets.

While numerous optimization algorithms have been developed to tackle various variants of the GQAP decision problem, this study introduces a pioneering approach known as the genetic engineering algorithm (GEA), which represents a modification of the traditional GA [23]. While Sohrabi et al. [23] applied the GEA to the vehicle routing problem (VRP) using benchmark instances, this study represents the first attempt to tailor the GEA for the GQAP.

The primary objective of this study is to augment solution quality and efficiency in resolving the GQAP. A conventional GA typically initializes the population of solutions randomly and iteratively refines these solutions using its basic search operators, such as crossover and mutation. However, its reliance on randomness often results in the premature convergence, susceptibility to local optima, and the generation of suboptimal solutions. To overcome these challenges, our research presents the GEA framework inspired by genetic engineering

principles. This framework incorporates three search scenarios that emulate gene mining processes, aiming to enhance population diversity and solution exploration. This GEA dynamically evolves chromosomes to exhibit desired characteristics through various techniques, such as insertion, purification, and genetic material exchange from the elite list of solutions. As a result, it effectively improves solution quality and accelerates convergence speed of the traditional GA using mutation and crossover operators. The proposed GEA offers a promising avenue for addressing the complexities of the GQAP as a complex mixed-integer nonlinear program. In conclusion, the main research highlights of this paper are as follows:

- The introduction of the GEA, a novel modification of the GA, to address the GQAP.
- The development of innovative search scenarios aimed at enhancing the population through gene exchange mechanisms.
- Experimental evaluations demonstrating the superior performance of the GEA across various search scenarios against state-of-the-art algorithms and exact solutions for benchmark problem instances.
- A comprehensive sensitivity analysis of the GEA's search scenarios, aiming to compare the GEA operators against traditional mutation and crossover operators.

The remainder of this paper is structured as follows: In Sect. 2, we delve into the problem definition and settings pertinent to our considered GQAP. Section 3 explores the intricacies of the proposed GEA, emphasizing the design and implementation details of genetic engineering operators. Section 4 delineates our experimental setup and conducts a thorough comparative analysis of the performance of our GEA against other variants and the traditional GA. Finally, in Sect. 5, we conclude by underscoring the significance of the modified GEA as an innovative solution for the GQAP while suggesting potential avenues for future research in this domain.

2 Generalized quadratic assignment problem (GQAP)

In this study, we focus on the GQAP originally proposed by Lee and Ma [1]. This decision problem focuses on the tactical planning for the allocation of equipment units to the factories within a predefined planning horizon, such as a season or a year. It entails the presence of m factories and a variety of n equipment units. Let's denote the factories as i and k, where i and k belong to the set $I = \{1, 2, ..., m\}$, with $i \neq k$, and the distance between factory i and k is represented by d_{ik}. Similarly, the equipment units are indexed by j and l, with $j, l \in J = \{1, 2, ..., n\}$, and $j \neq l$. Here, f_{jl} signifies the flow between equipment units j and l. The quadratic interaction cost, denoted by q_{ijkl}, characterizes the expense incurred when assigning equipment j to factory i and equipment l to factory k. Specifically, we define q_{ijkl} to simultaneously consider the product of the flow between equipment units f_{jl} and the distance between factories d_{ik}, i.e., $q_{ijkl} = f_{jl}d_{ik}$.

Treating the problem as an assignment optimization problem, we introduce c_{ij} to represent the assignment cost for transporting equipment j to factory i. We should highlight that each equipment unit must be assigned to only one factory as the main assumption of this decision problem. In addition, each equipment unit j necessitates a certain amount of space denoted by a_{ij} to be installed at factory i. In this regard, a crucial constraint to consider is that each factory i has a finite capacity, b_i, representing the maximum allowable space available for equipment installation. To define an optimization model for this GQAP, we define a binary variable x_{ij} to assign equipment units to factories. These variables assume a value of 1 if equipment unit j is assigned to factory i, and 0 otherwise.

In summary, the GQAP seeks to optimize the allocation of equipment units to factories, considering both assignment and quadratic interaction costs, along with the assignment and capacity constraints imposed by the available space at each factory. In this regard, the formulation of the original GQAP as a mixed-integer nonlinear program can be presented as follows:

Generalized Quadratic Assignment Problem (GQAP)

$$\min_{\mathrm{GQAP}}(Z = \sum_{i\in I}\sum_{j\in J} c_{ij}x_{ij} + \sum_{i\in I}\sum_{j\in J}\sum_{k\in I}\sum_{l\in J, l\neq j} q_{ijkl}x_{ij}x_{kl}) \tag{1}$$

Subject to:

$$\sum_{i\in I} x_{ij} = 1 \quad \forall j \in J \tag{2}$$

$$\sum_{j\in J} a_{ij}x_{ij} \leq b_i \quad \forall i \in I \tag{3}$$

$$x_{ij} \in \{0,1\} \quad \forall i \in I, j \in J \tag{4}$$

Equation (1) presents the objective function encompassing both linear and quadratic costs, capturing the total expense associated with equipment assignment and interaction within factories. Equation (2) represents the assignment constraints, mandating that each equipment unit must be allocated to precisely one factory, thereby ensuring a singular and exclusive assignment for each equipment unit. Equation (3) constitutes the set of capacity constraints, safeguarding that each factory possesses adequate capacity to accommodate all the assigned equipment units. These constraints are vital to prevent overloading and ensure operational efficiency. Finally, Eq. (4) defines the feasible range of the binary variables.

The GQAP poses a significant challenge due to its inherent nonlinearity, often rendering exact solutions impossible even for small instances. While various linear models have been proposed for the GQAP, this study adopts the most efficient linear reformulation proposed by Fathollahi-Fard et al. [2].

As mentioned earlier, the second part of the objective function of the GQAP introduces nonlinearity. To linearize this part, we replace the term $x_{ij}x_{kl}$, with a new variable y_{ijkl}. By doing so, we achieve linearity in the objective function. Now, we need to add a set of constraints to enforce the model to consider the relationship between y_{ijkl} and $x_{ij}x_{kl}$. In this regard, the interchangeability property of multiplication allows us to use y_{klij} to replace $x_{ij}x_{kl}$, just as y_{ijkl} replaces $x_{kl}x_{ij}$ since $x_{ij}x_{kl} = x_{kl}x_{ij}$. Fathollahi-Fard et al. [2] specifically addressed the assignment constraint set presented in Eq. (2) to consider the relationship between y_{ijkl} and $x_{ij}x_{kl}$. To this end, this constraint set, comprising n constraints, is multiplied by x_{kl} to yield mn^2 constraints:

$$\sum_{i\in I} x_{ij}x_{kl} = x_{kl} \quad \forall j, l \in J, k \in I \tag{5}$$

In this regard, we can reformulate the above constraint set as the following constraints:

$$\sum_{i\in I} y_{ijkl} = x_{kl} \quad \forall j, l \in J, k \in I \tag{6}$$

$$\sum_{i\in I} y_{klij} = x_{kl} \quad \forall j, l \in J, k \in I \tag{7}$$

Based on this concept, one efficient linearization of the GQAP, as demonstrated by Fathollahi-Fard et al. [2], is outlined as the following mixed-integer linear program:

Linearized Generalized Quadratic Assignment Problem (LGQAP)

$$\min_{\text{LGQAP}}(Z = \sum_{i\in I}\sum_{j\in J} c_{ij}x_{ij} + \sum_{i\in I}\sum_{j\in J}\sum_{k\in I}\sum_{l\in J, l\neq j} q_{ijkl}y_{ijkl}) \tag{8}$$

Subject to: Constraints (2), (3), (4), (6), and (7)

$$y_{ijkl} \geq 0 \ \forall i, k \in I, j, l \in J \tag{9}$$

Indeed, the LGQAP model proposed by Fathollahi-Fard et al. [2] introduces additional constraints compared to the original GQAP model. Specifically, it incorporates $2mn^2 + n + m$ constraints in addition to mn binary variables and n^2m^2 non-binary variables. While this augmented complexity may seem daunting, it is a necessary trade-off for achieving linearity, enabling easier solution procedures than the original nonlinear GQAP.

The original GQAP model comprises only $n + m$ constraints and mn binary variables, making it simpler in terms of constraint count. However, its nonlinear nature renders it computationally challenging, especially for larger problem instances. By transforming the problem into a linear model, the LGQAP model becomes more compatible with a broader array of commercial software packages, such as CPLEX and GAMS, which include mixed-integer linear programming solvers. Consequently, we can cross-validate our solutions obtained from the proposed GEA algorithm against the exact solutions derived from this linear model.

3 Proposed algorithm

In this section, we provide an overview of our approach to optimizing the GQAP. We begin by defining the chromosome representation for our optimization model (Sect. 3.1). Following that, we delve into the original GA employed (Sect. 3.2). Finally, we introduce our customized GEA procedures specifically tailored to address the complexities inherent in the GQAP (Sect. 3.3).

3.1 Chromosome definition

In the GQAP, our search space encompasses all feasible and infeasible solutions that conform to the assignment constraints delineated in Eq. (2), explored through the utilization of search operators within our metaheuristic algorithms. In the context of GAs, we define a solution as a "*chromosome*", serving as the representation for the GQAP solutions, as illustrated in Fig. 1. The size of each chromosome is based on the number of equipment units n, where each gene shows a unique number based on all the available number of factories m. In this regard,

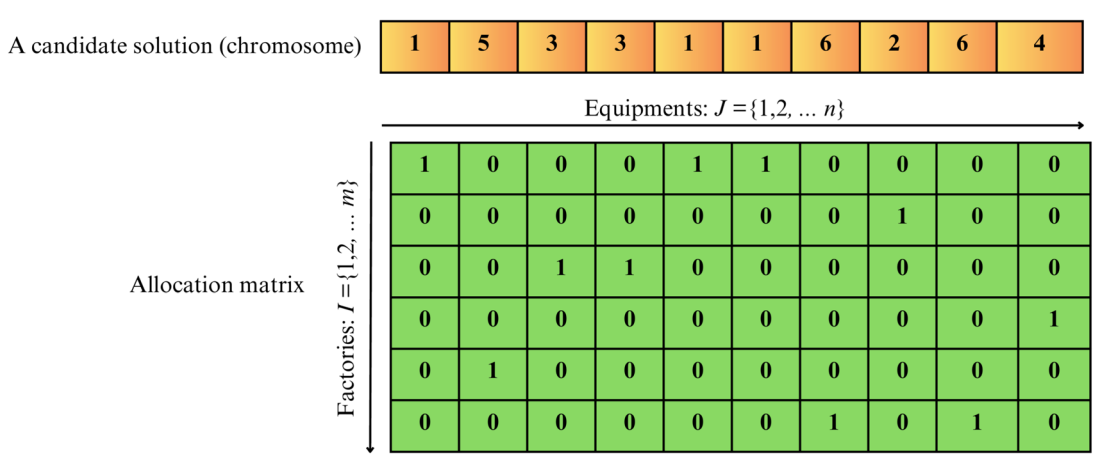

Fig. 1 Chromosome representation

individual genes represent the equipment allocated to a specific factory, thereby ensuring adherence to assignment constraints.

Subsequently, we construct the allocation matrix derived from the candidate solution or chromosome. If this allocation matrix satisfies the capacity constraints outlined in Eq. (3) of the GQAP, we proceed to compute the objective function detailed in Eq. (1). However, if the allocation matrix violates the capacity constraints, we assign an extremely large penalty value to the objective function, effectively excluding such solutions from the feasible search space. The metaheuristic algorithms in this paper follow this strategy to search for feasible and effective solutions within the constraints of the GQAP.

3.2 Traditional GA

Holland's pioneering work on the GA [24] stands as a seminal contribution to evolutionary and population-based approaches for tackling combinatorial optimization problems [25]. Drawing inspiration from biological genetics, the GA operates through two fundamental mechanisms, the crossover and mutation operators, which systematically explore the search space of the GQAP to discover promising solutions. In this study, we explore two candidate crossover operators: single-point and double-point crossovers [26], exemplified in Fig. 2. For each generation of the main algorithm, for the stage of the crossover process, our algorithm randomly selects one type of crossover to explore new solutions. The pseudo-codes for the single-point and double-point crossover operators are, respectively, shown in Algorithms 1 and 2.

Algorithm 1 One-point Crossover

Inputs: Parent Chromosomes p_1, p_2
Outputs: Offspring Chromosomes χ_1, χ_2

$n \leftarrow$ length of chromosome
$\Gamma \leftarrow$ Randomly select 1 crossover point from $\{1, 2, ..., n-1\}$
$\chi_1 \leftarrow [p_1(1:\Gamma), p_2(\Gamma+1:n)]$
$\chi_2 \leftarrow [p_2(1:\Gamma), p_1(\Gamma+1:n)]$
return χ_1, χ_2

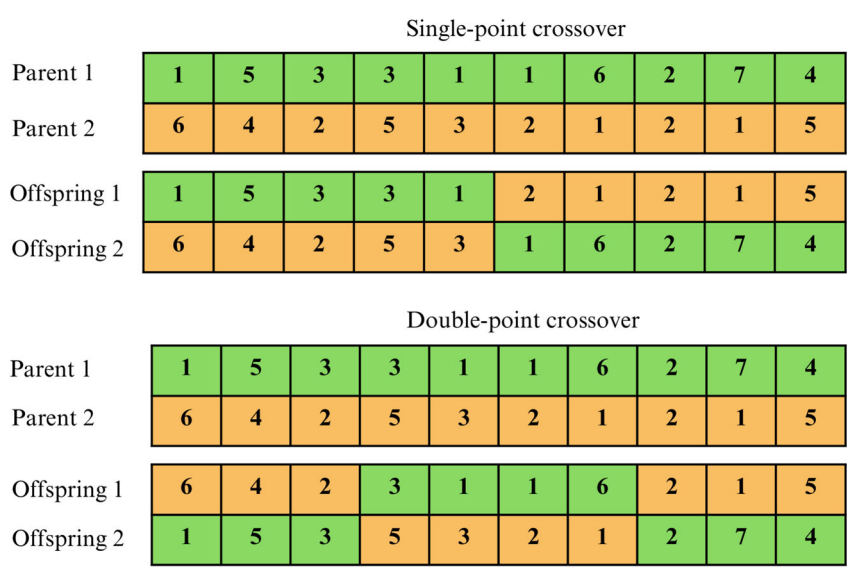

Fig. 2 Crossover variants applied to the GQAP

Algorithm 2 Two-point Crossover

Inputs: Parent Chromosomes p_1, p_2
Outputs: Offspring Chromosomes χ_1, χ_2

$n \leftarrow$ length of chromosome
$\Gamma_1, \Gamma_2 \leftarrow$ Sort(Randomly select 2 crossover points from $\{1, 2, ..., n-1\}$)
$\chi_1 \leftarrow [p_2(1:\Gamma_1), p_1(\Gamma_1+1:\Gamma_2), p_2(\Gamma_2+1:n)]$
$\chi_2 \leftarrow [p_1(1:\Gamma_1), p_2(\Gamma_1+1:\Gamma_2), p_1(\Gamma_2+1:n)]$
return χ_1, χ_2

Additionally, we consider five distinct mutation operators: swap, big swap, reversion, insertion, and random mutation [27], showcased in Fig. 3. Similar to the crossover stage, our algorithm randomly chooses one mutation type for each generation, fostering the exploitation of local-optimum solutions and adaptability in the search space. The pseudo-codes for the swap, big swap, reversion, insertion, and random mutation operators are, respectively, shown in Algorithms 3, 4, 5, 6, and 7.

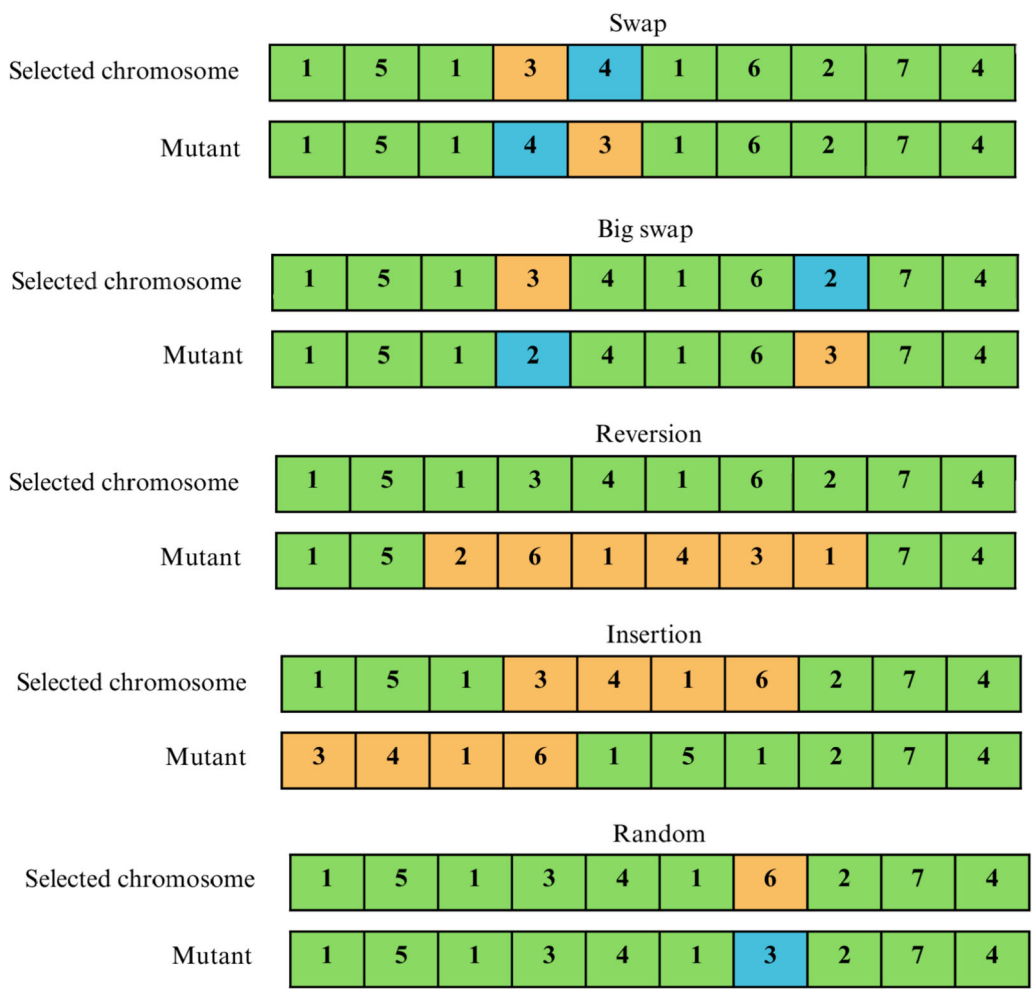

Fig. 3 Mutation variants applied to the GQAP

Algorithm 3 Swap Mutation

Inputs: Chromosome p
Outputs: Mutated Chromosome χ

$n \leftarrow$ length of chromosome
$\Gamma \leftarrow$ Randomly select 1 swap point from $\{1, 2, ..., n-1\}$
Swap $p(\Gamma)$ and $p(\Gamma + 1)$
$\chi \leftarrow p$
return χ

Algorithm 4 Big Swap Mutation

Inputs: Chromosome p
Outputs: Mutated Chromosome χ

$n \leftarrow$ length of chromosome
$\Gamma_1, \Gamma_2 \leftarrow$ Randomly select 2 distinct indices from $\{1, 2, ..., n\}$
Swap $p(\Gamma_1)$ with $p(\Gamma_2)$
$\chi \leftarrow p$
return χ

Algorithm 5 Reversion Mutation

Inputs: Chromosome p
Outputs: Mutated Chromosome χ

$n \leftarrow$ length of chromosome
$\Gamma_1, \Gamma_2 \leftarrow$ Sort(Randomly select 2 indices from $\{1, 2, ..., n\}$)
Reverse $p(\Gamma_1 : \Gamma_2)$
$\chi \leftarrow p$
return χ

Algorithm 6 Insertion Mutation

Inputs: Chromosome p
Outputs: Mutated Chromosome χ

$n \leftarrow$ length of chromosome
$\Gamma_1, \Gamma_2 \leftarrow$ Sort(Randomly select 2 indices from $\{2, ..., n\}$)
$temp \leftarrow p(\Gamma_1 : \Gamma_2)$
$prefix \leftarrow p(1 : \Gamma_1 - 1)$
$suffix \leftarrow p(\Gamma_2 + 1 : n)$
$\chi \leftarrow [temp, prefix, suffix]$
return χ

Algorithm 7 Random Mutation

Inputs: Chromosome p, Model M
Outputs: Mutated Chromosome χ

$n \leftarrow$ length of chromosome
$k \leftarrow$ RandomInteger(1, 5)
$\Gamma \leftarrow$ RandomSample(1, $n-1$, k)
while $i < k$ **do**
  $r \leftarrow$ RandomInteger(1, $M.I$)
  $p(\Gamma_i) \leftarrow r$
  $i \leftarrow i + 1$
end
$\chi \leftarrow p$
return χ

3.3 Proposed GEA

We employ the GEA inspired by the work of Sohrabi et al. [23], who initially introduced the concept of extracting patterns from elite population individuals to enhance the GA solutions for benchmarked VRP instances. Building upon this idea, we leverage it to refine the traditional GA for addressing the GQAP through three search scenarios inspired by genetic engineering. Our GEA, inspired by the genetic engineering concept, commences with a randomly initialized population and incorporates a selection stage where chromosomes are randomly chosen as parents based on the population limit for crossover and mutation operations. It further includes mutation and crossover operators and three search scenarios based on genetic engineering (see Fig. 4). While the crossover and

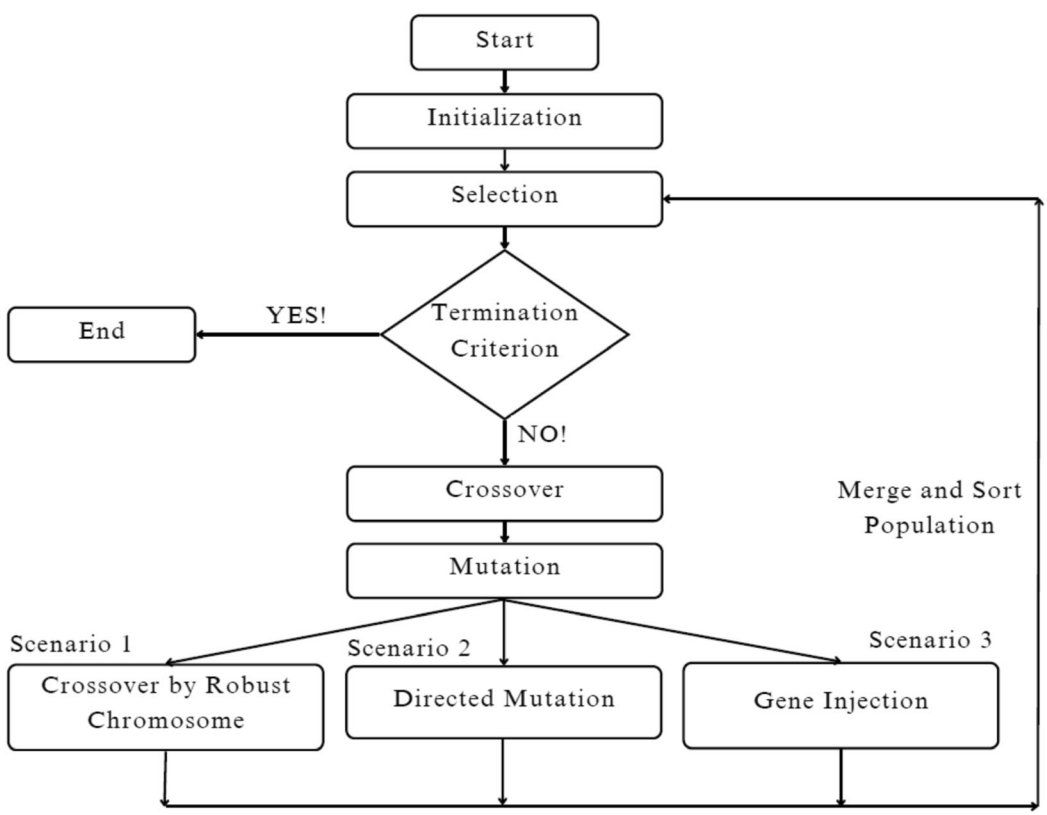

Fig. 4 Flowchart of the GEA

mutation operators are inherited from classical GAs, we introduce three innovative search scenarios, i.e., robust chromosome for crossover, directed mutation, and gene injection to the weakest chromosomes.

In the first novel operator, as one of the search scenarios of the GEA, we sequentially identify pairs of highly repeated patterns from the top $p\%$ of the population, referred to as the elite population. Using these patterns as masks, we construct a robust chromosome. The crossover operator is then performed by selecting this robust chromosome as one parent, while the second parent is chosen randomly from the population. This search scenario aims to integrate genes from the robust chromosome with multiple members of the population, fostering diversity and exploration.

The second proposed search scenario of the GEA employs the pattern matrix as a mask for the mutation operator. In this search scenario, a chromosome is randomly selected from the elite population, and a directed mutation is applied exclusively to non-informative genes, while informative genes remain unchanged. In this scenario, informative genes are those present in the mask, whereas non-informative genes are those absent from it.

Finally, in the third search scenario, we utilize the mask to inject informative genes into the remaining $(1-p)\%$ of the population, referred to as the non-elite population. This process involves selecting two chromosomes: one from the non-elite population and another from the elite population. The informative genes from the elite chromosome are then injected into the non-elite chromosome, enhancing its quality while maintaining genetic diversity.

These innovative operators as the search scenarios of the GEA collectively enhance the exploration and exploitation capabilities of our GEA, aiming to efficiently navigate the search space and converge to high-quality solutions for the GQAP. The flowchart of our GEA is shown in Fig. 4. We designate each innovative search operator as a search scenario. In the following sections, we provide a comprehensive definition and explanation of these search scenarios.

3.3.1 Search scenario 1

The first search scenario of the GEA primarily focuses on selecting the best chromosomes, specifically those with a higher fitness value, i.e., a lower total cost in the GQAP, compared to others. In this scenario, a user-defined percentage, denoted as $p\%$, is used to determine the selection of these top-performing chromosomes, collectively referred to as the *elite population*. The value of p is initially set by the user and can be fine-tuned to enhance the performance of the GEA.

A chromosome is considered *robust* if it exhibits the highest frequency of repeated gene pairs among the top $p\%$ of the elite population. If multiple chromosomes contain the same frequently repeated genes, one is randomly selected to dominate. However, if no fixed genes are available to create the mask, the algorithm, by default,

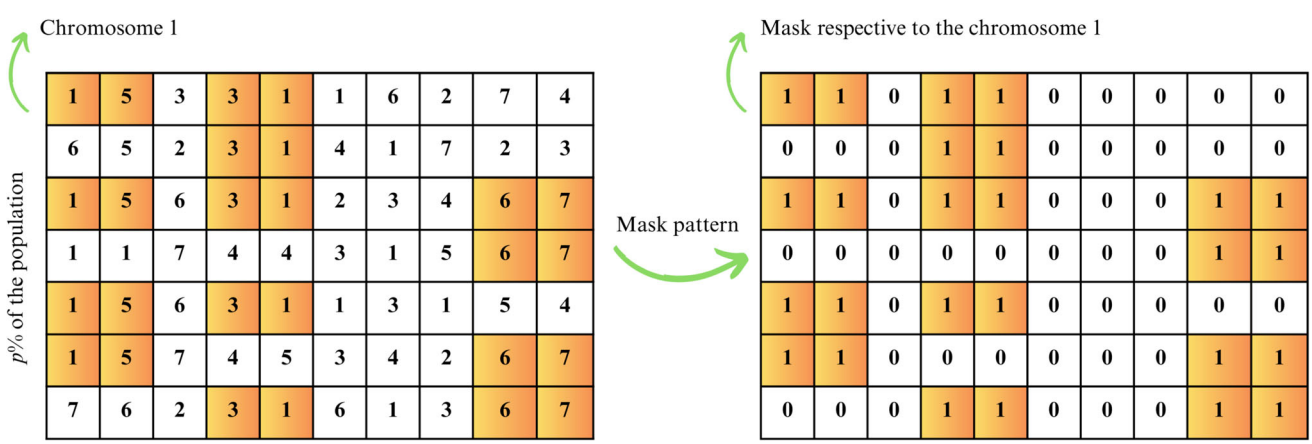

Fig. 5 Finding robust genes from $p\%$ of population, i.e., elite population

designates the first chromosome in the elite population as the robust chromosome and applies the first search scenario. Notably, the first chromosome in the elite population has the best fitness value, meaning it achieves the lowest total cost for the GQAP among the selected elite chromosomes.

Using the patterns of repeated gene pairs as masks, the robust chromosome is constructed. The crossover operator is then performed by selecting this robust chromosome as a fixed parent while randomly choosing the second parent from the population. To execute the crossover, one of the two crossover types, as previously described in Sect. 3.2, is randomly selected.

To illustrate this search scenario, consider Fig. 5, which provides an example of identifying the robust chromosome. As depicted in Fig. 5, if the threshold for pair repetition is set to 4, we observe that pairs (1, 5) and (6, 7) repeat exactly 4 times, while pair (3,1) repeats 5 times. Consequently, these gene pairs are classified as *informative genes* within their respective mask matrices, where they are assigned a value of 1 to denote their significance. Conversely, genes that are not present in the mask are referred to as *non-informative genes* and are assigned a value of 0 in the mask.

To further illustrate the encoding procedure for this search scenario, Algorithm 8 presents the pseudo-code detailing the construction of a robust chromosome. Additionally, Algorithm 9 outlines the crossover operator, demonstrating how the robust chromosome is combined with a randomly selected chromosome from the population, providing a structured guide for implementation.

Algorithm 8 Robust Chromosome

```
Inputs: Population Pop, Problem Info.
Outputs: Mask, Robust Chromosome DC

while row less than Pop size do
    while column less than chromosome length do
        temp ← 0
        while inner_row less than Pop size do
            if row equal to inner_row then
                continue
            end
            if Pop(row, column) equal to Pop(inner_row, column) then
                if Pop(row, column+1) equal to Pop(inner_row, column+1) then
                    temp ← temp + 1
                end
            end
            inner_row ← inner_row + 1
        end
        if temp grater and equal to threshold then
            Mask(row, column) ← 1
            Mask(row, column + 1) ← 1
            column ← column + 2
        else
            column ← column + 1
        end
    end
    row ← row + 1
end
DC ← chromosome with highest fixed gene in Mask
```

Algorithm 9 Crossover by Robust Chromosome

Inputs: Population Pop, Scenario_1 Crossover Rate C_r, Robust Chromosome DC
Outputs: Population Scenario_1 P_{sc1}

while i *less than* C_r **do**
  $Parent_2 \leftarrow$ *select chromosome randomly from* Pop
  $P_{sc1}(i) \leftarrow ApplyCrossover(DC, Parent_2)$
  $i \leftarrow i + 1$
end

3.3.2 Search scenario 2

In the second search scenario of our GEA, we focus on enhancing the effectiveness of the mutation operator, which plays a critical role in preventing the algorithm from getting trapped in local optima. Traditional GAs often rely on random mutation, which can introduce inefficiencies [27, 28]. To address this limitation, our GEA refines the mutation operator by introducing a directed mutation approach to improve algorithmic efficiency.

In this scenario, we employ specific strategies to guide the mutation process rather than relying on random selection. One such strategy involves identifying desirable genes, allowing the algorithm to prioritize those known to contribute to favorable traits or outcomes. By directing the mutation process and reducing the randomness associated with traditional mutation, the overall performance of the GEA can be significantly improved in comparison with the traditional GAs.

To illustrate the directed mutation process, Fig. 6 provides an example demonstrating its functionality. This mutation operator prioritizes specific genes, enabling beneficial modifications in a controlled manner. To perform this mutation, we examine the genes of a selected chromosome from the elite population. The informative genes, as identified in the mask from the first search scenario, remain unchanged. For the remaining non-informative genes, we apply one of the five mutation operator types, as detailed in Sect. 3.2.

By incorporating this directed mutation in the second search scenario, our refined algorithm overcomes the limitations of random mutation, ensuring more effective exploration and exploitation of the search space. For a detailed breakdown of the coding procedure for this directed mutation approach, refer to Algorithm 10.

Algorithm 10 Directed Mutation

Inputs: Population Pop, Mask, Mutation Rate Scenario_2 M_r
Outputs: Population Scenario_2 Pop_{sc2}

while i *less than* M_r **do**
  $temp \leftarrow$ *select chromosome randomly*
  $Pop_{sc2}(i) \leftarrow$ *Apply mutation on temp only on gene* $== 0$ *in* $Mask$
  $i \leftarrow i + 1$
end

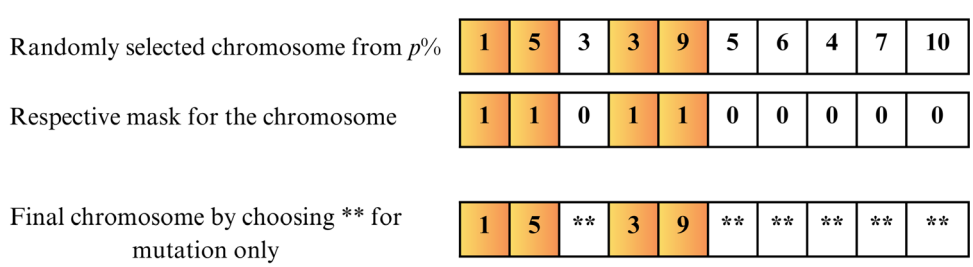

Fig. 6 Mutation directed by stabilizing informative genes

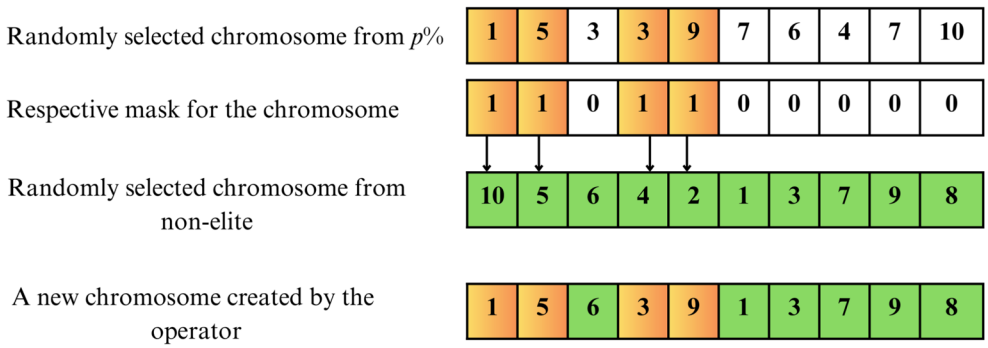

Fig. 7 Introducing informative genes into the weakest solutions of the population

3.3.3 Search scenario 3

The third search scenario within our proposed GEA underlines the significance of putting emphasis on the entire population and considering individual chromosomes which are suboptimal solutions. While the initial two search scenarios prioritize the elite portion of the population, it's imperative to acknowledge that even the poorest solutions can contribute to the overall algorithmic search enhancement. In many optimization algorithms, disregarding the worst individuals is unwise, as they harbor the potential for beneficial transformations [28, 29]. In our GEA's third search scenario, the goal is to reconsider these suboptimal individual chromosomes, employing a genetic engineering approach. By introducing subtle adjustments to these individuals, they can progress toward the more promising solutions and potentially join the elite solutions in subsequent algorithmic generations.

To achieve this, we construct a pattern matrix for the elite segment of the population. In each iteration of this scenario, one individual from the non-elite portion of the population (representing $(1-p)\%$) and one from the elite segment (representing $p\%$) are chosen. Utilizing the pattern matrix derived from the elite population, as explained in the first search scenario, gene pairs with the highest repetition across chromosomes are incorporated into the selected individual.

This gene injection operator streamlines the transfer of advantageous genetic information from the elite population to other individual chromosomes. Figure 7 illustrates an example of this proposed gene injection operator, showcasing how genetic information is transferred to enhance the selected chromosome. The mask matrix, delineating informative gene pairs in the elite population, plays a pivotal role in this scenario. Detailed insights into the coding process for this search scenario are provided in the pseudo-code outlined in Algorithm 11.

Algorithm 11 Gene Injection

Inputs: Population Pop, Mask, Gene Injection Rate Scenario_3 M_r
Outputs: Population Scenario_3 P_{sc3}

while i *less than* M_r **do**
  $non_elit \leftarrow$ *choose chromosome from nonelit population*
  $elit \leftarrow$ *choose chromosome from elit population*
  $P_{sc3} \leftarrow$ *inject genes to* non_elit *from elit where* $Mask == 1$
  $i \leftarrow i + 1$
end

Table 1 Tuned values of the GA and GEA's parameters

Algorithms		Parameters	Values
GEA	GA	Maximum number of generations	1000
		Population size	350
		Crossover rate	0.7
		Mutation rate	0.3
	Genetic engineering operators	p% of Scenario 1	0.3
		p% of Scenario 2	0.3
		p% of Scenario 3	0.5
		Fixing gene threshold	0.9
		Crossover rate in Scenario 1	0.5
		Mutation rate in Scenario 2	0.2
		Gene injection rate in Scenario 3	0.2

4 Experimental results

Here, we conduct an analysis of our algorithm on various GQAP benchmark instances, drawing from both small instances provided by Cordeau et al. [29] and a large-scale dataset sourced from Fathollahi-Fard et al. [2]. Complete datasets and algorithmic codes are available in the electronic supplementary materials accompanying the online version of this paper. For consistent analysis, the exact solver using GAMS software 24.7.4 was run on a PC equipped with an Intel(R) Core(TM) i7-10850H CPU @ 2.70 GHz 2.71 GHz processor, while the meta-heuristic algorithms were coded in MATLAB software R2013a and executed on a supercomputer at National Research University Higher School of Economics (HSE) University.

Initially, we fine-tuned the input parameters of our proposed algorithm and its variants (see Sect. 4.1 for more details). Subsequently, we juxtaposed the performance of our proposed algorithm against the state-of-the-art methods, demonstrating its efficacy and efficiency (see Sect. 4.2). Finally, we conducted a comprehensive sensitivity analysis of the key search scenarios within our GEA (see Sect. 4.3).

4.1 Parameter calibration analysis

Ensuring a fair comparison among different algorithms necessitates the establishment of a level playing field where each algorithm has an equal chance of discovering promising solutions. To achieve this parity, we have harmonized several critical parameters across all variations of the GA and our GEA, including the maximum number of generations, population size, crossover rate, and mutation rate. The determination of suitable values for these parameters involved a meticulous calibration analysis conducted using a full factorial method. In this calibration process, we drew upon the parameter ranges established in seminal works by Fathollahi-Fard et al. [2] and Chmiel [16]. Interested readers are encouraged to refer to the supplementary materials accompanying the online version of this paper for a comprehensive insight into the methodology and findings of the parameter selection process. In Table 1, we present the finalized values of the calibrated parameters, ensuring transparency and reproducibility in our experimental setup. These values serve as the cornerstone of our comparative analysis, enabling a robust assessment of the performance and efficiency of each algorithm variant across diverse benchmark instances.

4.2 Comparison with the GA-based algorithms

In our comprehensive analysis, we present a detailed comparison of the algorithmic performance and efficiency, as outlined in Table 2. The benchmark instances utilized encompass a range of complexities, with small instances denoted from c201535 to c351595 sourced from Cordeau et al. [29], and large instances denoted from E1 to E14

Table 2 Results of the algorithms based on the criteria of the minimum = Min, maximum = Max, average = Avg, and standard deviation = Std, where the best values in each criterion and test instance are highlighted in bold

Instance	Size ($n \times m$)	Exact Solution	GA				GEA_1				GEA_2			
			Max	Min	Avg	Std	Max	Min	Avg	Std	Max	Min	Avg	Std
c201535	20 × 15	6425.31	8111.57	**7085.98**	7201.25	219.841	7415.58	**7085.98**	7129.92	113.96	7415.58	**7085.98**	7134.1	110.32
c201555	20 × 15	15,547.8	24,564.4	**16,868.08**	20,780.1	1961.21	23,844.95	**16,868.08**	20,339.78	1884.9	23,269.18	**16,868.08**	20,447.93	2028.04
c201575	20 × 15	15,584.46	23,682.29	18,385.78	21,158.25	2010.81	25,963.83	18,385.78	22,227.77	1666.05	23,682.29	18,385.78	21,502.18	1911.3
c300695	30 × 6	47,342.29	63,328.57	53,295.03	56,762.53	2889.66	58,395.81	**52,669.85**	54,482.57	2278	63,306.77	**52,669.85**	54,726.74	2779.66
c300775	30 × 7	62,868.53	103,071.39	71,174.13	77,756.51	5659.03	110,908.83	71,174.13	77,796.35	6737.09	83,158.25	71,174.13	76,638.36	2514.61
c302055	30 × 20	28,756.27	49,326.28	36,336.85	39,421.74	3871.2	45,614.64	34,410.65	37,537.72	2640.71	47,112.19	**34,334.3**	38,450.76	2924.42
c351535	35 × 15	9524	15,422.73	**11,868.33**	13,029.33	1005.75	15,437.75	**11,868.33**	13,399.55	842.81	14,465.02	**11,868.33**	13,254.31	863.24
c351595	35 × 15	15,482.71	21,151.5	19,387.94	19,931.89	728.61	22,016.08	**18,505.87**	19,836.06	959.64	22,016.08	19,387.94	20,159	782.75
E1	100 × 5	OM	1,649,129.38	1,402,213.56	1,480,043.67	57,665.08	1,631,505.07	1,395,758.39	1,478,075.24	53,248.28	1,704,456.36	1,373,098.7	1,491,560.89	87,746.36
E2	200 × 5	OM	6,028,899.79	**5,611,155.16**	5,789,030.37	126,264.58	8,098,250.98	6,557,294.22	7,126,650.66	492,961.8	7,876,726.02	6,177,923.2	6,781,319.68	396,478.12
E3	100 × 10	OM	1,184,126.32	**1,121,614.81**	1,148,389.17	17,575.82	1,380,920.97	1,187,166.39	1,268,026.76	64,081.35	1,307,937.24	1,140,103.54	1,214,799.34	40,320.01
E4	200 × 10	OM	7,027,161.03	6,682,319.96	6,779,406.98	101,980.82	7,306,859.89	6,895,238.18	7,133,676.03	108,238.5	7,265,350.47	**6,672,065.08**	6,969,850.78	160,971.63
E5	400 × 10	OM	34,427,409.3	33,941,348	34,136,009.7	140,195.52	35,123,029.9	34,314,633.6	34,661,306.4	217,903.7	34,678,141.3	**33,941,076.6**	34,379,853.8	184,755
E6	900 × 15	OM	183,018,960	182,715,523	182,826,612	87,862.48	183,030,478	182,472,392	182,798,140	168,686.8	182,968,567	182,413,475	182,748,345	128,216.05
E7	100 × 20	OM	1,749,967.15	1,523,987.99	1,611,390.79	61,096.53	1,930,246.33	1,670,142.35	1,784,080.32	76,630.54	1,837,756.76	1,523,329.93	1,677,547.08	87,366.12
E8	200 × 20	OM	7,999,229.45	7,904,050.36	7,944,267.02	30,193.77	8,121,676.37	7,992,786.6	8,073,251.84	33,945.49	8,054,586.04	**7,889,119.36**	7,972,942.88	48,045.59
E9	400 × 20	OM	34,951,922.9	34,814,606.7	34,889,607.5	38,915.43	35,013,458.1	34,844,330	34,956,484.4	46,155.92	34,995,260.2	**34,748,175.8**	34,891,036.6	55,778.48
E10	900 × 30	OM	191,402,775	191,127,154	191,307,565	67,302.71	191,424,111	191,040,859	191,286,397	93,802.55	191,467,101	191,141,901	191,347,138	72,913.46
E11	400 × 40	OM	38,258,085.1	38,049,860.8	38,151,505.4	61,123.1	38,306,011.5	38,113,626.6	38,215,610	53,305.44	38,260,749.3	38,113,485.1	38,196,725.7	40,476.54
E12	900 × 60	OM	185,559,153	185,324,481	185,451,668	79,912.49	185,518,309	**185,226,070**	185,431,141	75,913.23	185,543,060	185,253,010	185,417,703	94,046.36
E13	1600 × 20	OM	578,318,883	578,055,044	578,200,699	64,448.11	578,274,969	**577,928,048**	578,146,438	89,090.49	578,271,691	578,037,013	578,164,433	59,644.14
E14	1600 × 40	OM	644,989,526	644,606,402	644,763,231	117,687.96	644,998,997	644,677,599	644,846,589	105,557.31	644,966,896	644,381,061	644,705,382	188,161.26
Number of the best-found solutions	–	–	–	5	–	–	–	7		–	–	9	–	–
Number of the unique best-found solutions	–	–	–	2	–	–	–	2		–	–	5	–	–

Instance	Size ($n \times m$)	Exact Solution	GEA_3				GEA			
			Max	Min	Avg	Std	Max	Min	Avg	Std
c201535	20 × 15	6425.31	7415.58	**7085.98**	7114.4	87.33	7415.58	**7085.98**	7136.37	115.44
c201555	20 × 15	15,547.8	24,527.33	**16,868.08**	20,656.98	2111.13	23,694.03	**16,868.08**	20,569.75	2150.13
c201575	20 × 15	15,584.46	23,682.29	**17,911.01**	21,109.04	2329.68	24,503.39	18,385.78	21,630.42	2305.99
c300695	30 × 6	47,342.29	58,246.17	**52,669.85**	54,432.65	2215.95	62,562.23	**52,669.85**	54,184.15	2494.98
c300775	30 × 7	62,868.53	82,226.85	**69,853.87**	76,204.67	2950.57	83,982.73	71,174.13	76,542.57	2924

https://doi.org/10.1007/s00521-025-11155-z

Table 2 (continued)

Instance	Size ($n \times m$)	Exact Solution	GEA_3				GEA			
			Max	Min	Avg	Std	Max	Min	Avg	Std
c302055	30 × 20	28,756.27	47,112.19	34,410.64	37,466.28	2851.38	47,096.49	35,624.7	38,170.37	3161.69
c351535	35 × 15	9524	15,555.58	**11,868.33**	13,264.15	1072.44	15,422.73	**11,868.33**	13,120.58	822.59
c351595	35 × 15	15,482.71	20,909.33	19,387.94	20,015.25	745.38	22,827.43	**18,505.87**	20,161.14	1164.33
E1	100 × 5	OM	1,616,177.59	**1,367,493.4**	1,478,131.89	49,290.06	1,659,490.67	1,409,295.8	1,524,580.7	76,176.58
E2	200 × 5	OM	7,508,480.71	6,386,225.6	6,910,072.76	252,720.16	8,783,652.64	7,999,875.52	8,428,419.95	215,021.21
E3	100 × 10	OM	1,359,731.46	1,180,839.34	1,253,775.15	48,074.65	1,535,791.71	1,364,456.93	1,441,303.65	43,923.06
E4	200 × 10	OM	7,273,351.2	6,735,313.53	7,030,657.87	151,135.62	7,635,242.78	7,271,563.82	7,466,209.65	108,365.99
E5	400 × 10	OM	34,764,643.1	34,097,621.9	34,453,525.9	175,827.8	35,343,310	34,680,390.8	35,021,381.1	190,242.79
E6	900 × 15	OM	182,966,307	**182,219,534**	182,769,286	154,791.8	183,051,412	182,631,500	182,875,174	92,803.27
E7	100 × 20	OM	1,862,657.44	**1,509,958.5**	1,669,517.05	88,833.81	2,044,668.89	1,702,386.13	1,899,548.97	74,071.56
E8	200 × 20	OM	8,046,448.27	7,929,700.21	7,986,572.19	35,279.4	8,206,210.27	8,076,483.05	8,141,223.43	31,164.8
E9	400 × 20	OM	34,990,072.6	34,875,743.4	34,916,086	28,560.88	35,035,812.3	34,929,532.3	34,979,503.9	31,281.14
E10	900 × 30	OM	191,421,893	191,173,306	191,313,118	76,527.99	191,434,756	**190,973,527**	191,319,994	96,985.15
E11	400 × 40	OM	38,244,179	**38,006,475.1**	38,161,023.8	69,820.87	38,298,761.2	38,111,459.5	38,213,030.1	49,859.63
E12	900 × 60	OM	185,580,890	185,239,638	185,445,923	95,720.93	185,541,470	185,292,985	185,425,895	83,055.29
E13	1600 × 20	OM	578,301,208	577,983,638	578,148,255	98,272.92	578,280,298	578,017,986	578,164,287	72,785.32
E14	1600 × 40	OM	645,082,057	**644,325,905**	644,792,535	220,062.33	644,951,657	644,497,159	644,787,680	135,107.14
Number of the best-found solutions	–	–	–	11	–	–	–	6	–	–
Number of the unique best-found solutions	–	–	–	7	–	–	–	1	–	–

Table 3 Comparison of CPU time (in seconds) and optimality gap = OG

Instance	Size (n × m)	Exact	GA		GEA_1		GEA_2		GEA_3		GEA	
		CPU	OG	CPU	OG	CPU	OG	CPU	OG	CPU	OG	CPU
c201535	20 × 15	17.12	0.10	163.59	0.10	333.20	0.10	296.03	0.10	296.17	0.10	603.44
c201555	20 × 15	120.84	0.08	150.53	0.08	329.96	0.08	289.80	0.08	291.28	0.08	591.33
c201575	20 × 15	465.78	0.17	151.02	0.17	320.71	0.17	282.51	0.14	272.45	0.17	565.54
c300695	30 × 6	170.83	0.13	66.71	0.11	264.12	0.11	250.72	0.11	246.20	0.11	646.06
c300775	30 × 7	1599.37	0.13	100.19	0.13	319.52	0.13	288.33	0.11	337.51	0.13	810.76
c302055	30 × 20	3600	0.26	567.20	0.19	1000.76	0.19	810.44	0.19	823.85	0.23	1000.97
c351535	35 × 15	3600	0.24	429.46	0.24	910.24	0.24	674.82	0.24	676.62	0.24	1000.29
c351595	35 × 15	3600	0.25	451.69	0.19	775.00	0.25	674.12	0.25	670.24	0.19	1000.93
E1	100 × 5	OM	–	357.41	–	1000.67	–	1000.05	–	986.14	–	1001.01
E2	200 × 5	OM	–	1000.84	–	1000.67	–	1000.11	–	1001.59	–	1001.14
E3	100 × 10	OM	–	1000.90	–	1002.19	–	1001.43	–	1002.09	–	1001.98
E4	200 × 10	OM	–	1002.46	–	1008.39	–	1001.90	–	1000.98	–	1010.25
E5	400 × 10	OM	–	1022.72	–	1007.30	–	1000.27	–	1001.73	–	1024.71
E6	900 × 15	OM	–	1136.36	–	1266.02	–	1301.67	–	1325.68	–	1315.06
E7	100 × 20	OM	–	1003.35	–	1006.48	–	1005.47	–	1004.44	–	1006.67
E8	200 × 20	OM	–	1002.29	–	1024.68	–	1011.65	–	1020.37	–	1007.93
E9	400 × 20	OM	–	1020.76	–	1005.71	–	1094.39	–	1079.88	–	1130.13
E10	900 × 30	OM	–	1178.94	–	1584.37	–	1319.61	–	1284.39	–	1826.50
E11	400 × 40	OM	–	1279.49	–	1060.34	–	1281.28	–	1433.43	–	1359.08
E12	900 × 60	OM	–	4952.24	–	6250.11	–	4975.00	–	5078.61	–	7581.41
E13	1600 × 20	OM	–	1594.58	–	2148.11	–	1917.41	–	1902.66	–	2722.88
E14	1600 × 40	OM	–	6464.00	–	9715.31	–	7911.46	-	7040.62	–	10,705.96

sourced from Fathollahi-Fard et al. [2]. It is essential to note that we employed the linear version of the GQAP, as detailed in Sect. 2 of the manuscript. Since Fathollahi-Fard et al. [2] previously demonstrated the performance of the LGQAP, we exclusively used this linear version in our comparison and did not employ the original nonlinear model. The interested readers can refer to Fathollahi-Fard et al. [2] for a comparison between the LGQAP and original GQAP models.

It should be noted that all small instances were successfully solved within a one-hour time limit using the CPLEX solver from GAMS software. However, for the large-scale instances, exact solutions were unattainable due to the memory limit of the PC used in this research, resulting in an "*out of memory*" (OM) message from our commercial software.

In addition to the exact solver, we evaluated the performance of various algorithmic approaches, including the original GA, as well as different configurations of our GEA. The GEA_1 represents our GEA with only the first search scenario active, GEA_2 with only the second search scenario active, GEA_3 with only the third search scenario active, and GEA with all search scenarios available to contribute to the next generation. For rigorous analysis, each variant of the algorithm (GA, GEA_1, GEA_2, GEA_3, and GEA) was executed thirty times on each test instance. We report the minimum, maximum, average, and standard deviation of the objective function values obtained for each instance, providing insights into the algorithmic stability and robustness as given in Table 2.

The analysis of the minimum values found by the algorithms, as presented in Table 2, reveals interesting insights into their performance. Among the algorithms, GEA_3 achieved the best values for 11 instances, and it emerged as the unique best performer for 7 instances. Following this top performer, GEA_2 demonstrated its effectiveness by being the best method in 9 instances, although it was the unique best in 5 instances. The GEA_1

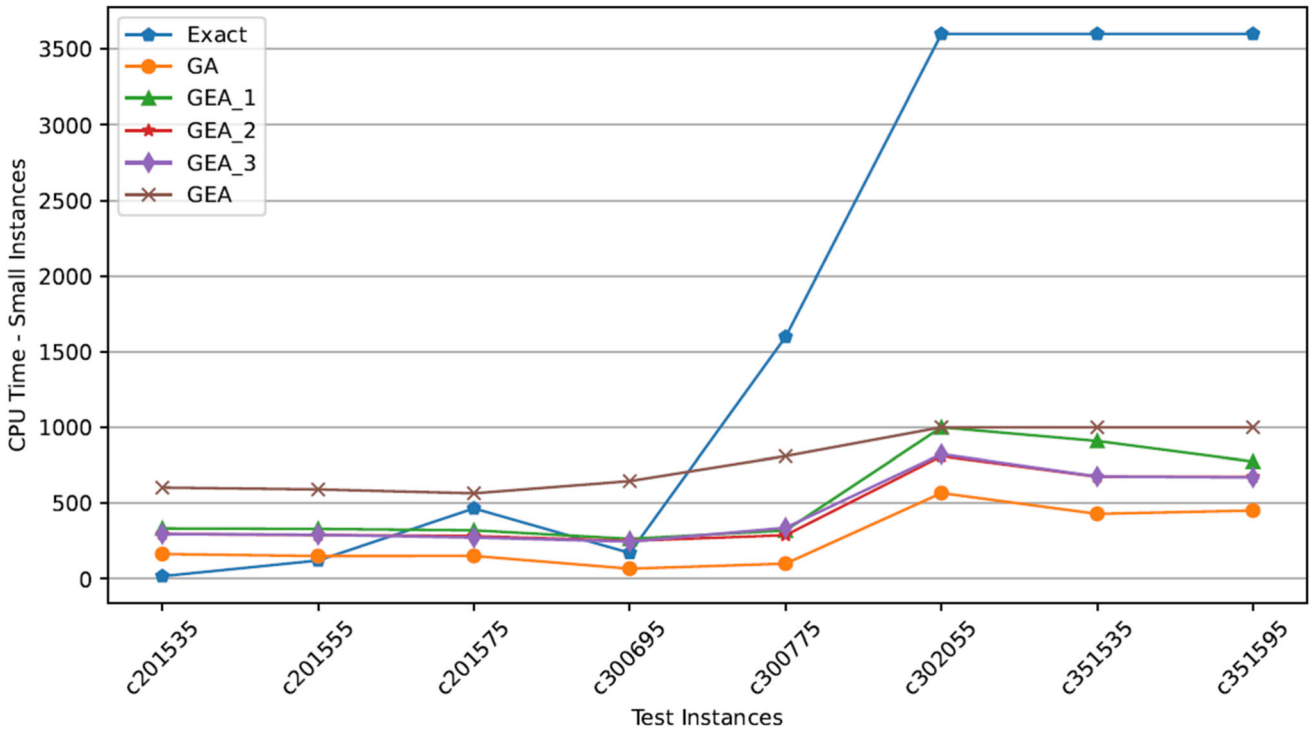

Fig. 8 Algorithms comparison based on the CPU time in small instances

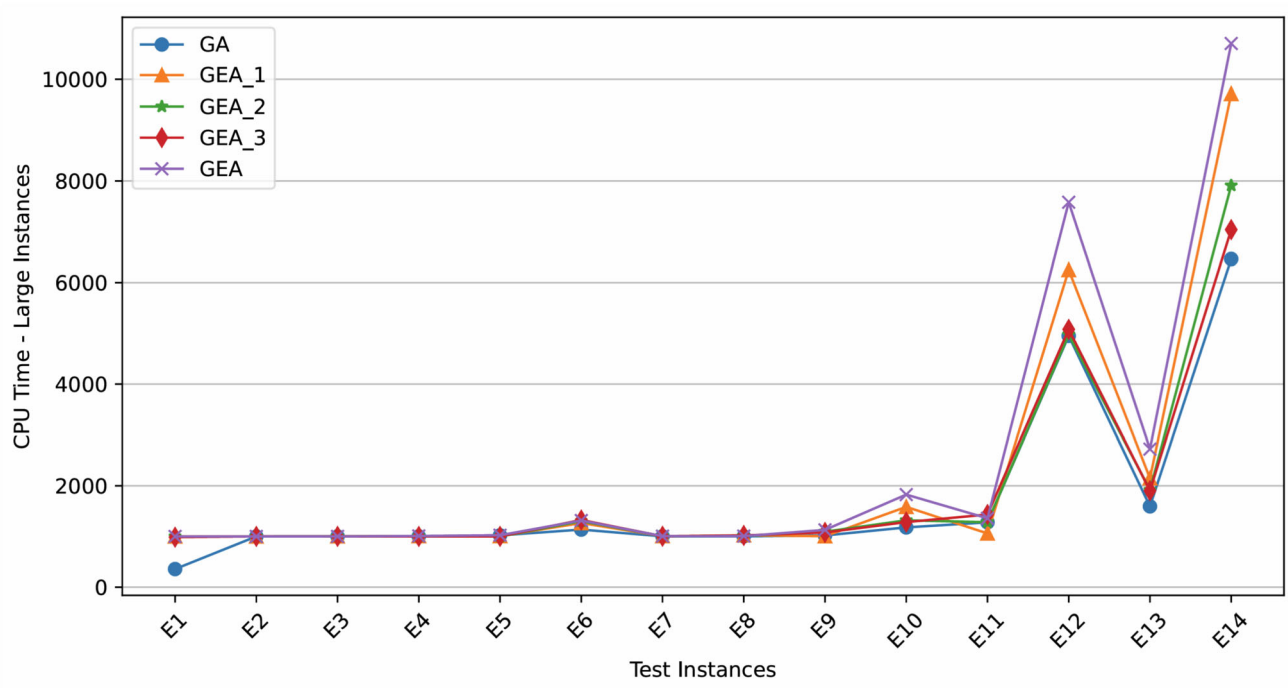

Fig. 9 Algorithms comparison based on the CPU time in large instances

outperformed both the GEA and GA, finding the best solutions in 7 instances, whereas the GEA was superior to the GA, achieving the best solutions in 6 instances. The GA achieved the best values for 5 instances while being the unique best performer for 2 instances only. These findings highlight the superior performance of the GEA_3 compared to the other GEA variants and the original GA in this comparison. By emphasizing the third search

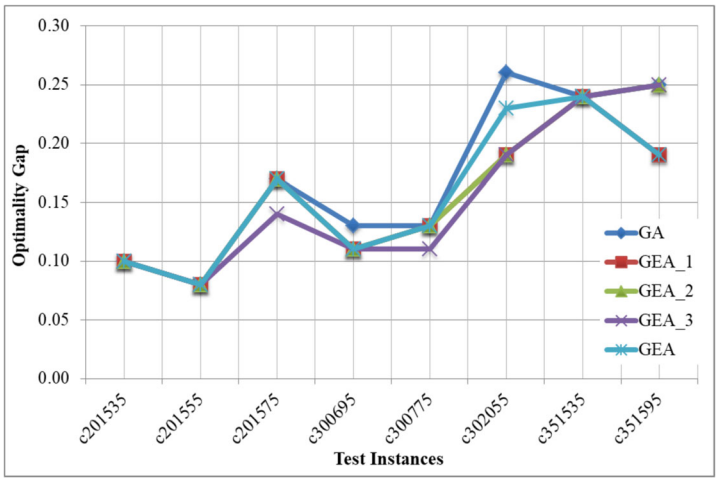

Fig. 10 Analyzing the optimality gap for the algorithms

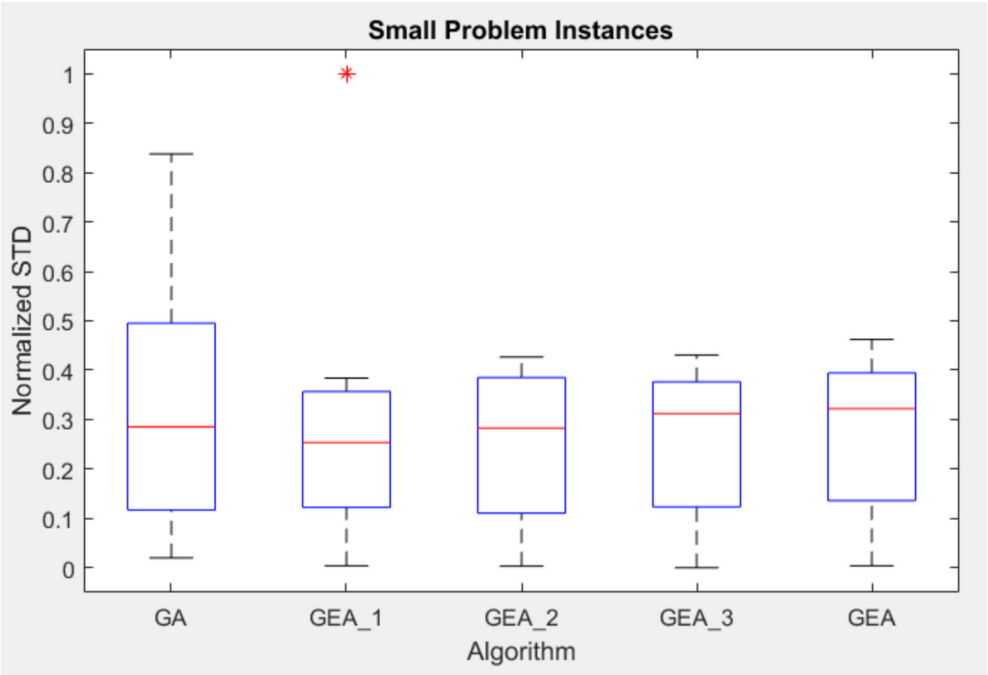

Fig. 11 Boxplot for the standard deviation of the algorithms in small instances

scenario, the GEA_3 showcases enhanced capabilities in finding good-quality solutions across a diverse range of instances. This underscores the efficacy of incorporating advanced search strategies, such as the targeted mutation approach and gene injection, in addressing the challenges posed by the GQAP.

Furthermore, Table 3 presents a comparative analysis of these algorithms based on CPU time in seconds and the optimality gap (OG) for each instance. To visually represent these findings, Figs. 8 and 9 depict the analyses of algorithms based on CPU time for small and large instances, respectively, while Fig. 10 showcases the comparison based on the OG for small instances. To further delve into the accuracy and robustness of the algorithms, we employ boxplots based on the statistical results provided in Table 2. Figures 11 and 12 illustrate the boxplots for the objective function standard deviation recorded for the candidate solution algorithms for small and large instances, respectively, offering a comprehensive visualization of algorithmic performance across multiple metrics.

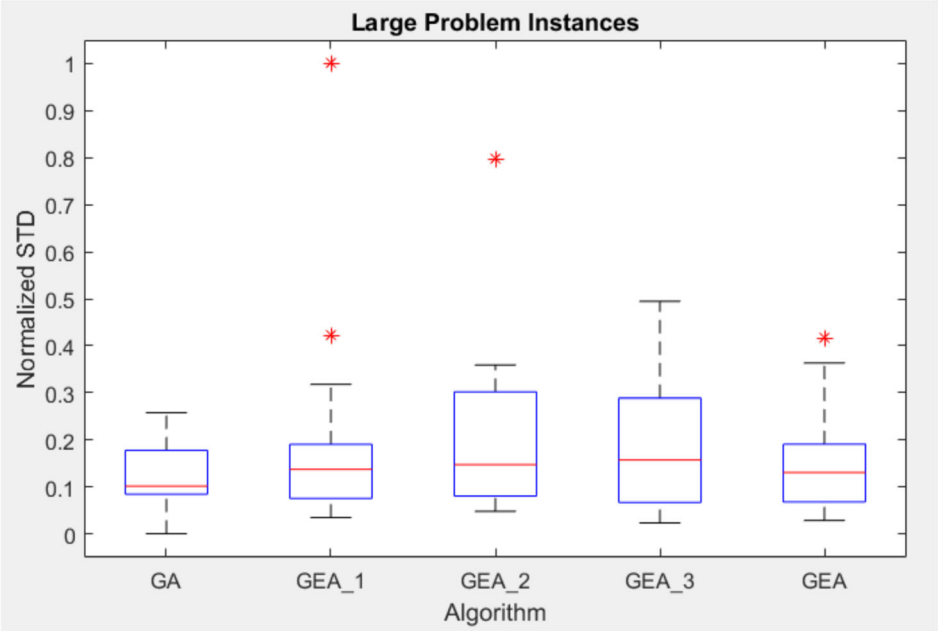

Fig. 12 Boxplot for the standard deviation of the algorithms in large instances

Based on the results reported in Table 3 and Fig. 8, it can be observed that the exact optimization approach outperformed all the GA and GEA variants only for two problem instances (c201535 and c201555), indicating its efficiency in specific scenarios. However, for instances ranging from c300775 to c351595, the exact solution proved to be prohibitively slow, rendering it impractical for these cases. Among the GA and GEA variants for small instances, the differences in CPU time are minimal. Interestingly, the GA exhibited slightly faster performance compared to the GEA variants, while the GEA variants generally required more time. Such a pattern in computational performance can be attributed to the additional steps involved in the three search scenarios within the GEA. While these steps enhance the algorithm's ability to explore the solution space comprehensively, they also entail additional computational overhead.

Referring to Fig. 9, depicting the comparison of algorithms for large-scale instances based on CPU time, we note a lack of significant difference between the performance of the GA and GEA variants. In addressing the large-scale instances of the GQAP, the GA and considered GEA variants exhibit comparable efficiency. However, our observation for the E12 and E14 instances confirms that the GEA requires more time compared to the GEA_1, while the other GEA variants do not impose a notable computational burden relative to the GA. Overall, the analysis of CPU time highlights the nuanced trade-offs between algorithmic complexity and performance. While the GEA variants may require slightly more computational resources due to their additional search scenarios, they offer enhanced exploration capabilities, particularly beneficial for large instances where exact solutions are not feasible.

The analysis of the OG criterion, depicted in Fig. 10, serves to validate the solutions generated by our GA and GEA variants for small instances. Note that the OG is only available for small instances because the exact solver failed to solve the large-scale instances, restricting our ability to compute the OG for our metaheuristic algorithms in those cases. Across the majority of small instances, with the exception of instances ranging from c302055 to c351595, the OG values for all algorithms are less than 0.20. This indicates a high level of confidence in the quality and accuracy of the solutions produced by our algorithms. Upon closer examination of the chart, we observe that the GEA_3 consistently achieves low OG values for small instances, demonstrating its superiority over other algorithms in terms of solution validation. Following the GEA_3, the GEA_1 emerges as the second-best algorithm, consistently achieving lower OG values compared to other variants. These findings underscore the effectiveness of our GEA, particularly when leveraging the advanced search strategies implemented in the

Table 4 Results of the Wilcoxon's test for analyzing the GEA variants against the GA for 30 independent runs ($\alpha = 0.05$)

Instance	GEA_1 in comparison with GA		GEA_2 in comparison with GA		GEA_3 in comparison with GA		GEA in comparison with GA	
	p-value	Win	*p*-value	Win	*p*-value	Win	*p*-value	Win
c201535	0.143	–	0.072	–	0.0481	+	0.284	–
c201555	0.583	–	0.673	–	0.7	–	0.721	–
c201575	0.124	–	0.492	–	0.785	–	0.391	–
c300695	0.965	–	0.951	–	1	–	0.474	–
c300775	0.918	–	0.632	–	0.289	–	0.432	–
c302055	0.144	–	0.728	–	0.107	–	0.481	–
c351535	0.086	–	0.466	–	0.478	–	0.91	–
c351595	0.795	–	0.284	–	0.208	–	0.275	–
E1	0.956	–	0.812	–	0.812	–	0.113	–
E2	1.91E − 06	+	1.91E − 06	+	1.91E − 06	+	1.91E − 06	+
E3	1.91E − 06	+	1.91E − 06	+	1.91E − 05	+	1.91E − 06	+
E4	1.91E − 06	+	9.54E − 06	+	1.91E − 06	+	1.91E − 06	+
E5	1.91E − 06	+	0.0003	+	9.54E − 06	+	1.91E − 06	+
E6	0.647	–	0.294	–	0.01	+	0.097	–
E7	1.91E-06	+	0.029	+	0.036	+	1.91E-06	+
E8	3.81E-06	+	0.044	+	0.004	+	3.81E-06	+
E9	0.0003	+	0.985	–	0.029	+	3.81E-06	+
E10	0.647	–	0.075	–	0.621	–	0.053	–
E11	0.001	+	0.647	–	0.003	+	0.0003	+
E12	0.267	–	0.241	–	0.903	–	0.541	–
E13	0.039	+	0.105	–	0.026	+	0.153	–
E14	0.322	–	0.375	–	0.322	–	0.769	–
Number of significant wins	–	9	–	6	–	11	–	8

GEA_3. By prioritizing solution quality and validation, the GEA_3 and GEA_1 showcase their capability to generate reliable solutions for small instances of the problem.

As part of experiments, we also conducted a series of statistical tests using boxplots to analyze the accuracy and robustness of the algorithms across 30 run times for each instance. To facilitate comparison, we normalized the standard deviation of the objective function values obtained by the considered algorithms within each instance. Two boxplots were defined for each solution approach for small and large instances, as depicted in Figs. 11 and 12. Based on the results obtained in Fig. 11, focusing on small instances, we observe that the typical GA generally yielded greater values of the standard deviation in the objective function values. On the contrary, GEA and its variants demonstrated a higher robustness level in the obtained results and higher accuracy. In the meantime, GEA, GEA_2, and GEA_3 generally had lower standard deviation values compared to GEA_1. Figure 12 presents the comparison of boxplots for the algorithms based on the normalized standard deviation from 30 run times for the large instances. Based on the conducted analysis, all the considered algorithms, including GA, demonstrated a high level of robustness. However, for several large problem instances, it was observed that GEA_1 and GEA_2 recorded greater values of the standard deviation in the objective function values. GEA_2 and GEA_3 exhibited a higher accuracy level compared to other algorithms.

To conduct a more robust statistical analysis of the algorithms, this study employs Wilcoxon's signed-rank test, which is widely regarded as one of the most reliable nonparametric statistical tests [30]. This test is applied by considering the best solutions obtained by each algorithm across 30 independent runs for each test instance of the GQAP using a significance level of 95% ($\alpha = 0.05$). The goal of this analysis is to statistically compare the

Table 5 Comparison of the best results obtained by this study with the ones reported in the relevant literature (the best values are shown in bold)

Instance	Fathollahi-Fard et al. [2]		This study	
c201535	**6854**	Found by ALNS	7085.98	Found by GA, GEA_1, GEA_2, GEA_3, GEA
c201555	17,271	Found by ALNS, TS, GA	**16,868.08**	Found by GA, GEA_1, GEA_2, GEA_3, GEA
c201575	**17,329**	Found by GRASP	17,911.01	Found by GEA_3
c300695	56,994	Found by ALNS	**52,669.85**	Found by GEA_1, GEA_2, GEA_3, GEA
c300775	79,894	Found by ALNS	**69,853.87**	Found by GEA_3
c302055	**31,592**	Found by ALNS	34,334.3	Found by GEA_2
c351535	72,521	Found by ALNS	**11,868.33**	Found by GA, GEA_1, GEA_2, GEA_3, GEA
c351595	102,313	Found by ALNS	**18,505.87**	Found by GEA_1, GEA
E1	2,854,281	Found by ALNS	**1,367,493.4**	Found by GEA_3
E2	12,260,749	Found by ALNS	**5,611,155.16**	Found by GA
E3	8,155,523	Found by GA	**1,121,614.81**	Found by GA
E4	51,467,162	Found by GA	**6,672,065.08**	Found by GEA_2
E5	225,660,071	Found by GA	**33,941,076.6**	Found by GEA_2
E6	1,255,372,932	Found by ALNS	**182,219,534**	Found by GEA_3
E7	30,405,509	Found by GA	**1,509,958.5**	Found by GEA_3
E8	97,674,403	Found by ALNS	**7,889,119.36**	Found by GEA_2
E9	642,167,938	Found by GA	**34,748,175.8**	Found by GEA_2
E10	4,322,757,423	Found by ALNS	**190,973,527**	Found by GEA
E11	1,366,272,223	Found by ALNS	**38,006,475.1**	Found by GEA_3
E12	7,156,723,055	Found by ALNS	**185,226,070**	Found by GEA_1
E13	6,655,675,298	Found by GA	**577,928,048**	Found by GEA_1
E14	16,548,175,691	Found by GA	**644,325,905**	Found by GEA_3

performance of each GEA variant against the original GA. The results of these statistical comparisons are summarized in Table 4. From the analysis, we observe that GEA_3 outperforms the other variants, achieving 11 wins across 22 instances. In comparison, other GEA variants show fewer wins, i.e., GEA_1 secured 9 wins, GEA_2 achieved 6 wins, and the original GEA attained 8 wins. These findings demonstrate the relative effectiveness of each algorithmic variant, with GEA_3 showing the strongest overall performance in comparison with other GEA variants.

In summary, the outcomes of the experiments conducted in this section for different GEA variants highlight the effectiveness of our search scenarios, particularly the third scenario, where informative genes are injected into the non-elite chromosomes. This approach significantly enhances the accuracy and robustness of the GA. By integrating these search scenarios, our GEA effectively explores the search space of the GQAP, leading to improved performance across a wide range of instances.

Last but not least, to evaluate the effectiveness of this study for analyzing the GQAP, we compare our results with the state-of-the-art solutions available in the literature. Specifically, we report the best solutions obtained using GA, GEA_1, GEA_2, GEA_3, and GEA and benchmark them against the results presented by Fathollahi-Fard et al. [2]. Since Fathollahi-Fard et al. [2] publicly shared all the GQAP instances used in their study, we were able to conduct a direct comparison under identical experimental conditions.

In the study by Fathollahi-Fard et al. [2], multiple metaheuristic algorithms, including ALNS, TS, GRASP, and GA, were employed to solve the GQAP. However, it is important to note that their GA differs from ours, as they utilized only two types of crossover and mutation operators, whereas our study incorporates two types of crossover operators and five types of mutation operators to enhance search diversification and intensification for the given metaheuristic algorithms.

Table 6 Sensitivity analyses of the GEA_3 on the test instance c201575, where Min represents the minimum and Avg represents the average values of the objective function recorded

Crossover operator			Mutation operator			Search scenario 3			
Crossover rate	Min	Avg	Mutation rate	Min	Avg	Gene injection rate	p%	Min	Avg
10%	18,385.78	21,948.74	10%	18,385.78	21,564.09	10%	10%	18,385.78	21,793.52
30%	21,002.8	22,668.11	30%	18,001.87	21,325.44	30%	20%	17,606.4	21,913.12
50%	18,385.78	23,025.26	50%	18,001.87	19,943.78	50%	30%	17,606.4	21,960.73
70%	18,385.78	22,905.08	70%	17,911.01	19,692.93	70%	40%	17,911.01	21,965.31

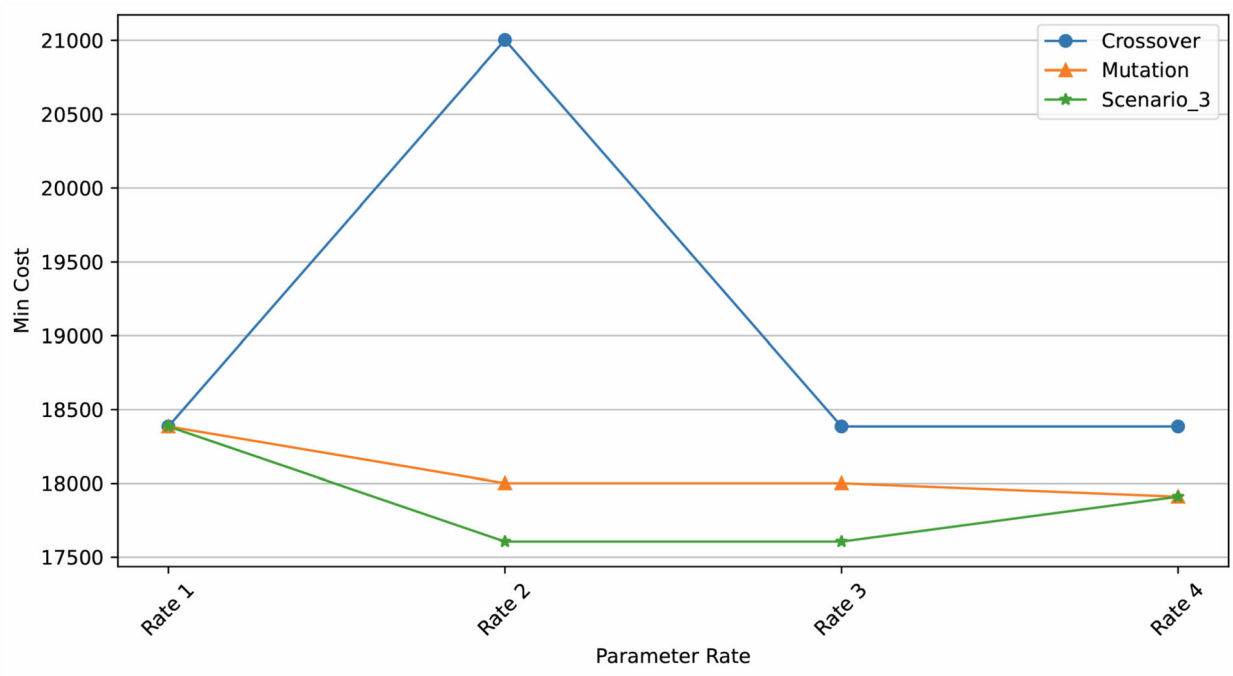

Fig. 13 Sensitivity analysis results for the minimum values found by the GEA_3 algorithm for different rates on crossover, mutation, and gene injection

The results presented in Table 5 are noteworthy, demonstrating the superiority of the results of this study in comparison with the results reported in the literature. Specifically, in the majority of test instances, our algorithms, particularly GEA_3, outperform the state-of-the-art methods. Notably, GEA_3 achieves the best solutions in 9 out of the 22 instances. The only exceptions are instances c201535, c201575, and c302055, where the ALNS and GRASP algorithms from Fathollahi-Fard et al. [2] yielded slightly better results. In all other cases, our approach delivers superior solutions, highlighting the effectiveness and robustness of the proposed method in solving the GQAP.

4.3 Sensitivity analyses

Here, we conduct in-depth sensitivity analyses on our best algorithm in this study, the GEA_3, which augments the original GA with the third search scenario. We selected this algorithm based on the minimum values found in Table 2, its performance for the OG in Fig. 10, and its high accuracy and robustness demonstrated in Figs. 11 and 12. When conducting the sensitivity analyses, we focused on a specific small instance, c201575. As previously

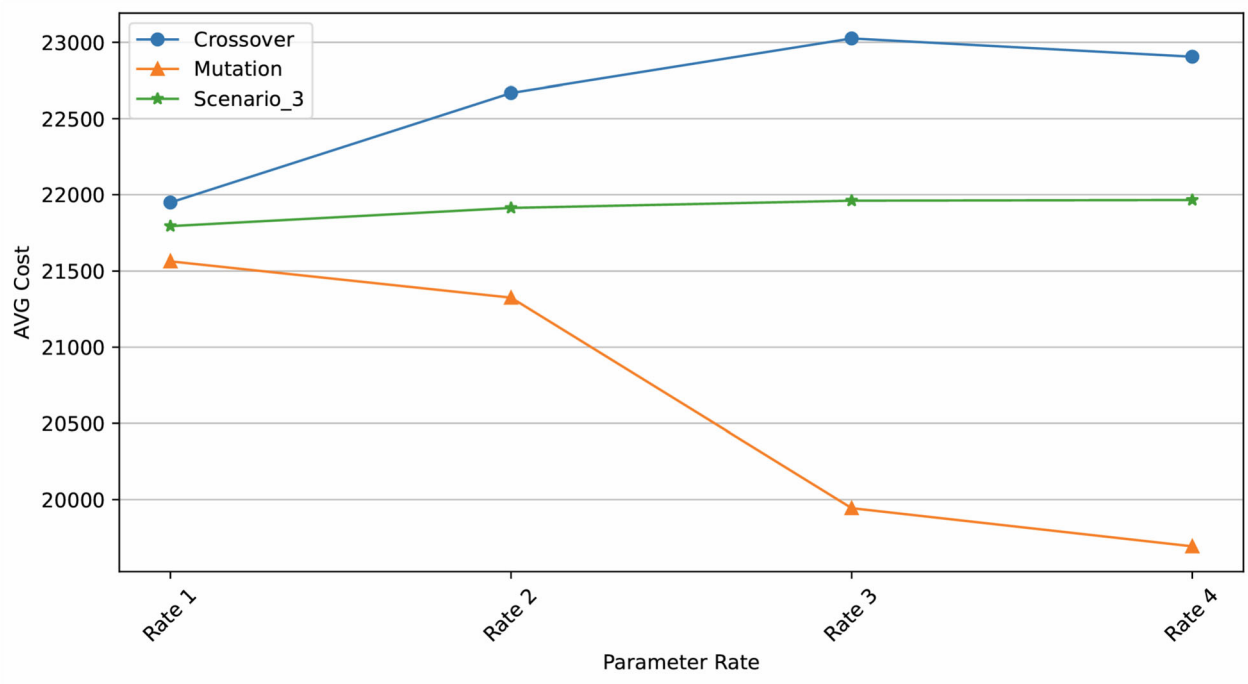

Fig. 14 Sensitivity analysis results for the average values found by the GEA_3 algorithm for different rates on crossover, mutation, and gene injection

explained, the search operators of this algorithm include crossover and mutation and the gene injection operator of the third search scenario. We varied the rates of these search operators while keeping the values of the other operators fixed at 10% according to the calibrated values provided in Table 1. The results of these sensitivity analyses are presented in Table 6. In each sensitivity analysis, the GEA_3 was executed 30 times to ensure the reliability of the results. We recorded both the minimum and average objective function values obtained. This approach allows us to rigorously evaluate the impact of different operator rates on the algorithm's performance and provide insights into the robustness and efficiency of the GEA_3 under varying conditions.

Figure 13 displays the minimum objective function values obtained from the algorithm for each search operator, while Fig. 14 illustrates the average objective function values obtained from the algorithm for each search operator across the sensitivity analyses. From the analysis of the minimum or best values, we observe that increasing the rate of mutation and the gene injection rate in the third scenario may initially enhance the algorithm's performance. Particularly, an increase in the gene injection rate in the third search scenario appears to contribute to improved success in finding promising solutions. However, this increase should ideally be within the range of 10–50 percent. A further increase in the gene injection rate may actually worsen the algorithmic performance. Conversely, increasing the crossover rate does not significantly impact the algorithm's performance. Based on the results reported in Fig. 14, it is evident that increasing the rate of the mutation operator only enhances the algorithm's performance in finding more local-optimum solutions, and the average objective values can potentially improve. The average objective function values were not substantially impacted after changing the crossover rate and the gene injection rate in the third search scenario.

In conclusion, our sensitivity analyses on the GEA_3 algorithm reveal valuable insights into the impact of varying search operator rates on the algorithm performance. Increasing the rates of mutation and the gene injection in the third search scenario within specified ranges shows promise in enhancing the algorithm's ability to find high-quality solutions. However, adjusting the crossover rate does not provide significant efficiency gains in terms of the algorithmic search capabilities. These findings underscore the importance of carefully calibrating the search operator rates to optimize the algorithmic performance for specific problem instances of the GQAP.

5 Conclusions and future works

In summary, this paper introduces a new metaheuristic algorithm, namely the genetic engineering algorithm (GEA), to address the challenging generalized quadratic assignment problem (GQAP). Inspired by genetic engineering principles, the GEA enhances traditional GA by introducing a sophisticated search mechanism, akin to gene mining processes. Through targeted insertion, purification, and genetic material exchange from elite solutions, the GEA dynamically evolves chromosomes to yield superior solution quality and convergence speed. Evaluation against state-of-the-art algorithms and exact solutions across diverse GQAP instances showcases the GEA's remarkable performance, particularly with the GEA_3 demonstrating superior results. By emphasizing the third search scenario when promising genes from elite individuals are injected into solutions with fairly low fitness, the GEA_3 exhibits enhanced capabilities in finding high-quality solutions, surpassing both the traditional GA and other GEA variants. Sensitivity analyses further confirm the effectiveness of the GEA_3, highlighting the impact of mutation and gene injective rate on the algorithmic performance.

Despite the GEA's effectiveness in enhancing the solution quality and convergence speed, the additional search mechanisms introduced in the GEA, such as the gene mining and targeted mutation, may lead to increased computational overhead. As a result, the GEA may face increasing computational time when applied to highly complex real-world problems with large datasets. Furthermore, while the GEA demonstrates superior performance compared to the traditional GA and other variants, there remains a need for further validation across a wider range of problem instances and benchmark datasets. Additionally, the reliance on predetermined parameter values, as calibrated in this study, may limit the adaptability of the GEA to different problem domains and datasets. Future research efforts could explore methods for automatically tuning the algorithmic parameters or adapting them dynamically during runtime to optimize performance across diverse problem instances.

Another future research suggestion could delve into the exploration of the GEA's search space dynamics, convergence behavior, and the interaction between its search mechanisms using adaptive learning mechanisms. Although the current study primarily focused on the application of the GEA to the GQAP, the principles underlying the GEA's design could be extended to other quadratic problems beyond the GQAP. Hence, our next research project could explore the applicability and effectiveness of the GEA in addressing a broader range of optimization problems in various domains, such as logistics, scheduling, and engineering design and applications.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s00521-025-11155-z.

Acknowledgements This article is the output of a research project implemented as part of the Basic Research Program at the National Research University Higher School of Economics (HSE University). This research was supported in part through the computational resources of HPC facilities at HSE University [31].

Data availability All data and codes are available in the supplementary materials accompanying the online version of this publication.

Declarations

Conflict of interest The authors declare that there are no conflicts of interest.

References

1. Lee, C. G., and Ma, Z. (2004). The generalized quadratic assignment problem. Research Rep., Dept., Mechanical Industrial Eng., Univ. Toronto, Canada, M5S.
2. Fathollahi-Fard AM, Wong KY, Aljuaid M (2023) An efficient adaptive large neighborhood search algorithm based on heuristics and reformulations for the generalized quadratic assignment problem. Eng Appl Artif Intell 126:106802

3. Martello, S., and Toth, P. (1990). Knapsack problems: algorithms and computer implementations. John Wiley and Sons, Inc.
4. Hochbaum, D. S. (2005). Complexity and algorithms for convex network optimization and other nonlinear problems. 4OR, 3(3), 171–216.
5. Wolpert DH, Macready WG (1997) No free lunch theorems for optimization. IEEE Trans Evol Comput 1(1):67–82
6. Cattrysse DG, Van Wassenhove LN (1992) A survey of algorithms for the generalized assignment problem. Eur J Oper Res 60(3):260–272
7. Laguna M, Kelly JP, González-Velarde J, Glover F (1995) Tabu search for the multilevel generalized assignment problem. Eur J Oper Res 82(1):176–189
8. Dıaz JA, Fernández E (2001) A tabu search heuristic for the generalized assignment problem. Eur J Oper Res 132(1):22–38
9. Yagiura M, Ibaraki T, Glover F (2006) A path relinking approach with ejection chains for the generalized assignment problem. Eur J Oper Res 169(2):548–569
10. Mateus GR, Resende MG, Silva RM (2011) GRASP with path-relinking for the generalized quadratic assignment problem. Journal of Heuristics 17:527–565
11. Liu L, Gao X (2009) Fuzzy weighted equilibrium multi-job assignment problem and genetic algorithm. Appl Math Model 33(10):3926–3935
12. Billionnet A, Elloumi S, Lambert A (2012) Extending the QCR method to general mixed-integer programs. Math Program 131(1):381–401
13. Billionnet A, Elloumi S, Lambert A (2013) An efficient compact quadratic convex reformulation for general integer quadratic programs. Comput Optim Appl 54(1):141–162
14. Robenek T, Umang N, Bierlaire M, Ropke S (2014) A branch-and-price algorithm to solve the integrated berth allocation and yard assignment problem in bulk ports. Eur J Oper Res 235(2):399–411
15. McKendall A, Li C (2017) A tabu search heuristic for a generalized quadratic assignment problem. J Ind Prod Eng 34(3):221–231
16. Chmiel W (2019) Evolutionary algorithm using conditional expectation value for quadratic assignment problem. Swarm Evol Comput 46:1–27
17. Bergman D (2019) An exact algorithm for the quadratic multiknapsack problem with an application to event seating. INFORMS J Comput 31(3):477–492
18. Guignard M (2020) Strong RLT1 bounds from decomposable Lagrangean relaxation for some quadratic 0–1 optimization problems with linear constraints. Ann Oper Res 286(1):173–200
19. D'Ambrosio C, Martello S, Monaci M (2020) Lower and upper bounds for the non-linear generalized assignment problem. Comput Oper Res 120:104933
20. Silva A, Coelho LC, Darvish M (2021) Quadratic assignment problem variants: A survey and an effective parallel memetic iterated tabu search. Eur J Oper Res 292(3):1066–1084
21. Zhou Q, Hao JK, Wu Q (2022) A hybrid evolutionary search for the generalized quadratic multiple knapsack problem. Eur J Oper Res 296(3):788–803
22. Song M, Cheng L (2022) Solving the reliability-oriented generalized assignment problem by Lagrangian relaxation and Alternating Direction Method of Multipliers. Expert Syst Appl 205:117644
23. Sohrabi M, Fathollahi-Fard AM, Gromov VA (2024) Genetic Engineering Algorithm (GEA): An Efficient Metaheuristic Algorithm for Solving Combinatorial Optimization Problems. Autom Remote Control 85:252–262. https://doi.org/10.1134/S000511792403007X
24. Holland JH (1973) Genetic algorithms and the optimal allocation of trials. SIAM J Comput 2(2):88–105
25. Srinivas M, Patnaik LM (1994) Genetic algorithms: A survey. Computer 27(6):17–26
26. Poon PW, Carter JN (1995) Genetic algorithm crossover operators for ordering applications. Comput Oper Res 22(1):135–147
27. Cui L, Li G, Lin Q, Chen J, Lu N (2016) Adaptive differential evolution algorithm with novel mutation strategies in multiple sub-populations. Comput Oper Res 67:155–173
28. Lin A, Sun W, Yu H, Wu G, Tang H (2019) Global genetic learning particle swarm optimization with diversity enhancement by ring topology. Swarm Evol Comput 44:571–583
29. Cordeau JF, Gaudioso M, Laporte G, Moccia L (2007) The service allocation problem at the Gioia Tauro maritime terminal. Eur J Oper Res 176(2):1167–1184
30. Jain M, Singh V, Rani A (2019) A novel nature-inspired algorithm for optimization: Squirrel search algorithm. Swarm Evol Comput 44:148–175
31. Kostenetskiy PS, Chulkevich RA, Kozyrev VI (2021) HPC Resources of the Higher School of Economics. J Phys: Conf Ser 1740(1):012050. https://doi.org/10.1088/1742-6596/1740/1/012050

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Majid Sohrabi[1,2] · Amir M. Fathollahi-Fard[3] · Vasilii A. Gromov[1] · Maxim A. Dulebenets[4]

✉ Amir M. Fathollahi-Fard
amirfard.ie@gmail.com

✉ Maxim A. Dulebenets
mdulebenets@eng.famu.fsu.edu

Majid Sohrabi
msohrabi@hse.ru

Vasilii A. Gromov
stroller@rambler.ru

1 School of Data Analysis and Artificial Intelligence, National Research University Higher School of Economics, 11 Pokrovsky Boulevard, Moscow, Russian Federation 109028

2 Laboratory for Models and Methods of Computational Pragmatics, National Research University Higher School of Economics, Moscow, Russian Federation

3 Department of Engineering Science, Faculty of Innovation Engineering, Macau University of Science and Technology, Taipa, Macau, China

4 Department of Civil & Environmental Engineering, Florida A&M University-Florida State University (FAMU-FSU) College of Engineering, 2035 E Paul Dirac Dr., Sliger Building, Suite 275, Tallahassee, FL 32310, USA